

single line:

```
def goToScene(self, nextSceneKey, data=None):  
    --- snip ---  
    self.oSceneMgr._goToScene(nextSceneKey, data)
```

This makes a call to the private `_goToScene()` method in the scene manager. Within the scene manager's method, we need to give the current scene an opportunity to do any cleanup that might be needed, then transfer control to the new scene. Here is the code of the `_goToScene()` method of the scene manager:

```
def _goToScene(self, nextSceneKey, dataForNextScene):  
    --- snip ---  
    if nextSceneKey is None: # meaning, exit  
        pygame.quit()  
        sys.exit()  
  
    # Call the leave method of the old scene to allow it to clean up.  
    # Set the new scene (based on the key) and  
    # call the enter method of the new scene.  
    ❶ self.oCurrentScene.leave()  
    pygame.key.set_repeat(0) # turn off repeating characters  
    try:  
        ❷ self.oCurrentScene = self.scenesDict[nextSceneKey]  
    except KeyError:  
        raise KeyError("Trying to go to scene '" + nextSceneKey +  
            "' but that key is not in the dictionary of scenes.")  
    ❸ self.oCurrentScene.enter(dataForNextScene)
```

The `_goToScene()` method performs a number of steps to transition from the current scene to the target scene. First, it calls `leave()` in the current scene ❶ so the current scene can do any necessary cleanup. Then, using the target scene key that was passed in, it finds the object for the target scene ❷ and sets that as the current scene. Finally, it calls `enter()` for the new current scene ❸ to allow the new current scene to do any required setup.

From this point on, the `run()` method of the scene manager loops and calls the `handleInputs()`, `update()`, and `draw()` methods of the current scene. These methods will be called in the current scene until the program executes